



Python Fundamentals I

Data Analytics Bootcamp



Topic

Understanding Python



Core Philosophy

Python follows 19 principles outlined in [“The Zen Of Python”](#). Here are 3 of the most **important**.

1. “Readability counts”
2. “Explicit is better than implicit”
3. “Simple is better than complex”



“Readability counts”

“**Readability counts**” means you should write code that's easy for humans to understand.

Use **clear** formatting and **descriptive** names so your code becomes self-documenting, making maintenance and collaboration simpler.



“Explicit is better than implicit”

“Explicit is better than implicit” reminds you to clearly show what your code is doing.

When you **spell out** operations directly rather than relying on hidden behaviors, you create **more predictable** code with fewer unexpected bugs.



“Simple is better than complex”

“Simple is better than complex” encourages you to choose straightforward solutions first.

When you avoid unnecessary complexity, your code becomes **easier** to understand and maintain. Add complexity only when it truly **adds value** to your solution.



Basic Syntactic Structure

1. Indentation creates **code blocks** (4 spaces)
2. Comments start with #
3. Whitespace matters for structure



A group of commands that work together and are marked by the same amount of space at the start of each line.

```
# This is a comment
```

```
def greet_user(name):
```

```
[ ]# Indentation to create code block
```

```
    message = "Hello, " + name
```

```
    return message
```

```
# Whitespace to structure for readability
```

```
user_name = "Jane"
```

```
greeting = greet_user(user_name)
```

```
if greeting.startswith("Hello"):
```

```
    print("Greeting is friendly!")
```

```
    print("The message says: " + greeting)
```

Basic Syntactic Structure



What Are Variables?

Variables are names for **values** in Python.

- Used anywhere a value or expression is used
- Assigned using the assignment operator =
- Allows us to store and manipulate data

```
user_name = "Jane"
```



Why Use Variables?

Variables are essential for **dynamic** programming.

- Store and update information
- Make code more readable and maintainable
- Allow for reuse of values throughout your code

```
user_name = "Jane"  
print("Hello, " + user_name)  
# Hello, Jane
```

```
user_name = "Duane"  
print("Hello, " + user_name)  
# Hello, Duane
```



Variable Declaration Syntax

Declaring a variable in Python tells Python to reserve memory for a value and defines the name that will be used to access it.

- **Name:** A unique identifier for the variable
- **Operator:** The “=” informs Python this is a variable.
- **Value:** The data being assigned to the variable

```
user_name = "Jane"
```

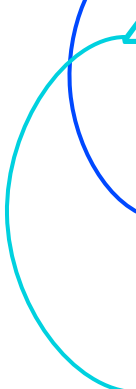


Naming Variables

1. Must start with a letter or underscore
2. Can contain letters, numbers, and underscores
3. **Case sensitive** “name” does not equal “Name”
4. Cannot use Python **keywords**



Case sensitivity means “name” and “Name” are as different as “name” and “pharmaceutical”



Keywords are specific words used to tell Python to complete specific tasks - they will be covered later on.



What Are Data Types?

Data types are categories for different kinds of **values** in Python:

- Define what operations can be performed on the data
- Determine how the data is stored in memory
- Prevent errors by ensure proper data handling

```
user_name = "Jane"
```



Python Data Types

Some Python data types are **self-explanatory**.

- **Numbers:** int, float (e.g. 42, 3.14)
- **String:** text (e.g. “Jane”)

Some will be covered in detail **later on**.

- **Boolean:** True / False values
- **Collections:** Stores multiple items together

What Are Operators?

Operators are symbols and phrases that tell Python to perform specific operations.

- **Arithmetic:** `+`, `-`, `/`, `%`, `**`, `//`
- **Assignment:** `=`, `+=`, `-=`, `*=`, `/=`
- **Comparison:** `==`, `!=`, `>`, `<`, `>=`, `<=`
- **Logical:** `and`, `or`, `not`

What Is An Expression?

An expression is a combination of values, variables, and operators that can be evaluated to produce a result.

```
print("Hello, " + user_name)  
# Hello, Duane
```

The **print** statement must be used in order for this result to show in the console.

It's important to note that Python will **always** evaluate expressions, but it will not show you the result unless it is **told** to do so.



Exercise

Basic Python Syntax